
FIVE HOURS, 3,649 REPOSITORIES: AN EMPIRICAL STUDY OF AN AGENT-ASSISTED MASS HACKATHON

PREPRINT

Talgar Bayan
[pending: affiliation]
talgar.bayan@gmail.com

September 27, 2026

ABSTRACT

HackAlem AI was a five-hour, in-person hackathon held in Astana, Kazakhstan, on 23 September 2026. The organisers created a public GitHub repository for every registration and required teams to use an AI coding agent (OpenAI Codex); the event was also organised as an official Guinness World Records attempt. Because every registration received a repository, the organisation’s 3,649 repositories record which registrations produced code, when, and with which tools, including those that produced none. We mirrored all repositories with their full history and studied participation, work rhythm, technology choices, traces of agent use, documentation language, and credential hygiene. Only 1,057 repositories (29.0%) received team commits during the event, and 65.6% received none at all. Most active teams committed in at least four of the five hours, yet 28.8% of event-time commits fell in the last hour, and a cluster of 30.6% of teams made most of their commits then. Test files appeared in 76.9% of repositories with team commits and agent context files in 31.0%. README files were mostly in Russian (87.0%); 2.9% were in Kazakh. Strings with the structure of OpenAI API keys appeared in the public history of 61 repositories, often in `.env.example` files. We release the derived data and the pipeline, and discuss what organisers of agent-assisted events can learn from these records.

Keywords hackathons · mining software repositories · AI coding agents · developer behaviour · secret leakage

1 Introduction

Hackathons are time-bounded events in which small teams build software prototypes, and they have been studied both through case studies and at scale through the repositories teams leave behind [1–3]. In parallel, AI programming assistants and, more recently, autonomous coding agents have changed how software is written. Controlled experiments and field studies report faster task completion with assistants [4, 5], observational work describes how programmers interact with them [6, 7], and agents are now evaluated on real repository tasks [8, 9] and observed authoring pull requests in open-source projects [10, 11]. Studies of generative AI inside hackathons exist, but they are qualitative and small: interviews at one event [12], one educational event with nine teams [13], a month-long online event [14], and a single university case [15].

HackAlem AI offers a different view. It took place in Astana on 23 September 2026, in person, with a five-hour coding window, teams of up to three people, and a requirement to use Codex during development [16]. It received 6,586 applications for 2,500 seats [17, 18] and was organised as an official Guinness World Records attempt for the most participants in an agentic AI hackathon, with the result pending at the time of writing [19]. For our purposes, the key design choice was technical: the organisers created a public repository in the BAITC-Hacks GitHub organisation for every registration and archived the repositories after the deadline. The organisation therefore contains every registration, not only the projects that teams chose to submit or list, and every team worked under the same deadline and tooling rule.

We mirrored all 3,649 repositories with their full history, rebuilt every measure from git, and checked the result against the GitHub API (Figure 1). We ask seven research questions:

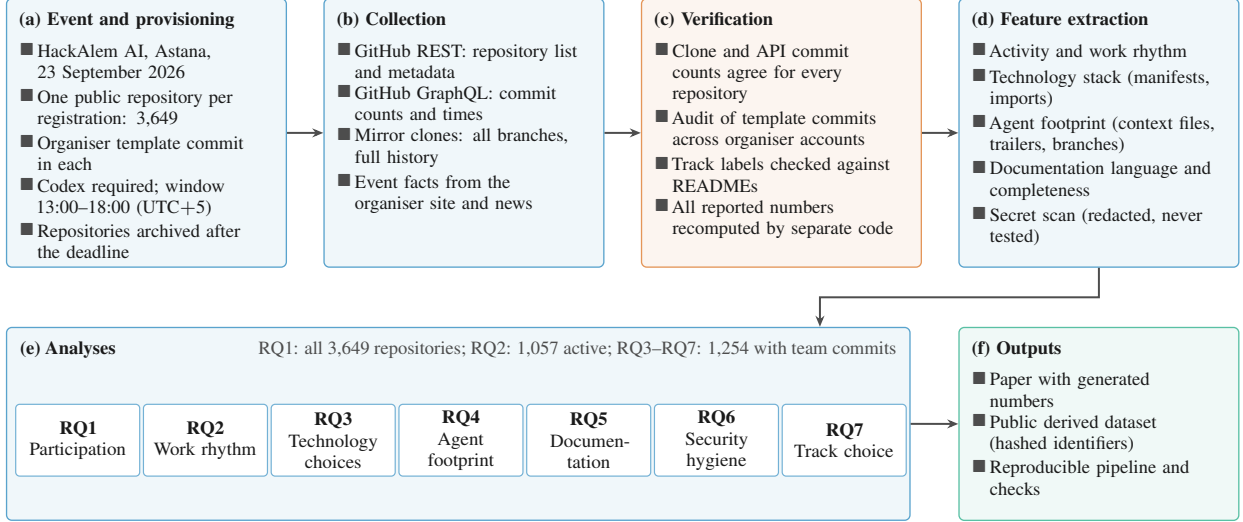


Figure 1: Overview of the study. Every registration received a public repository (a). We collected the repositories and their full history (b), checked the collected data against independent sources (c), and derived the features (d) used for seven research questions (e). The outputs (f) are this paper, a derived dataset with hashed identifiers, and the pipeline with its checks.

RQ1 How many registrations turned into code, and does this depend on when the repository was created?

RQ2 How did teams distribute their work over the five hours?

RQ3 Which languages, frameworks and LLM services did teams use?

RQ4 What traces of AI coding agents do the repositories contain?

RQ5 In which languages did teams document their work, and how complete is the documentation?

RQ6 How often did credentials reach the public history, and where?

RQ7 How did teams distribute over the 12 tracks?

To our knowledge, this is the first repository-level study of a complete hackathon population in which AI-agent use was required. Our contributions are:

- an empirical account of participation, work rhythm, technology choices, agent traces, documentation and credential hygiene across all 3,649 repositories of one event (Section 5);
- measurement lessons for mining organiser-created repositories, including template commits made by more than one organiser account and agent signatures that reflect tool defaults rather than use (Sections 4 and 6);
- a derived dataset with hashed identifiers and a reproducible pipeline in which every reported number is generated from the data (Section 8).

2 Background and related work

Hackathons and their repositories. Nolte et al. studied continuation of hackathon projects across many events and found that short-term continuation relates to preparation and winning, while long-term continuation relates to team skills [1]. McIntosh and Hardin analysed the GitHub repositories of the U.S. Major League Hacking events of two seasons and found that most commits happen within the first week and few projects remain active six months later [2]. HackRep provides a large dataset of hackathon repositories collected from project listings, with analyses of continuation, team composition and event geography [3]. These datasets contain projects that teams chose to list; they cannot show registrations that produced no code.

AI programming assistants and coding agents. A controlled experiment found that developers with GitHub Copilot finished a programming task faster than a control group [4], and a large user study related perceived productivity to usage data [5]. Programmers interact with such assistants in an acceleration mode and an exploration mode [6], and developers report both benefits and control problems [7]. Agents now operate on whole repositories: SWE-bench measures

Table 1: Facts about the event used in this paper. Organiser and news sources were accessed on 24 September 2026.

Item	Value	Source
Date and place	23 September 2026, Astana, in person	[16]
Coding window	five hours; 13:00–18:00 local time (UTC+5), inferred from commit and archive times	[19, 38]
Teams	up to three members; solo entry allowed	[16]
Tooling rule	Codex required during development	[16]
Capacity	2,500 seats; 6,586 applications	[16–18]
Participants	from 21 countries	[19, 38]
Prizes	US\$1.1 million in prizes and technology resources	[16]
Record attempt	Guinness World Records, most participants in an agentic AI hackathon; previous record 2,089 participants; result pending	[19]
Results	awards on 1 October 2026	[16]
Repositories	one public repository per registration in BAITC-Hacks; archived after the deadline	this study

issue resolution [8], SWE-agent studies agent–computer interfaces [9], and the AIDev dataset collects agent-authored pull requests from five agents [10]. Using AIDev, Agarwal et al. report velocity gains after agent adoption together with persistent quality risks [11]. Agent context files such as AGENTS.md and CLAUDE.md have been characterised in open-source repositories [20], and their effect on agent task success has been evaluated [21]. SWE-chat records real agent sessions and finds that in a large share of them the agent writes nearly all committed code [22].

Vibe coding and generative AI in hackathons and education. “Vibe coding” describes building software mainly through natural-language prompts to an AI system [13, 23]. Hackathon studies report that teams use generative AI for coding, learning and documentation and check its output under time pressure [12], that novices can deliver working demonstrations but engage little with engineering practice [13], and that a no-manual-editing rule shapes prompting and debugging [14]. Computing educators have reviewed the implications of these tools for teaching [24, 25].

Secrets in public repositories. Secret leakage on GitHub is frequent and persistent [26]; detection and mitigation methods have been proposed [27] and benchmarked [28]. Developers who leaked secrets describe the difficulty of remediation [29], and code completion models can reproduce hard-coded credentials [30]. Two recent lines of work concern AI-built software directly: LLM API keys leaked from mobile apps [31], and vulnerabilities in agent-generated and vibe-coded applications [32, 33].

Mining GitHub. GitHub data carries known perils, such as inactive or personal repositories and unmerged-looking pull requests [34], and different mining tools can disagree on basic counts [35]. README content has been categorised in earlier work [36]. Work on developer procrastination informs our reading of deadline effects [37].

3 The event and its infrastructure

Table 1 lists the facts about the event that we use, with their sources. Registration was free; solo registration was allowed [16]. News reports describe long queues at the entrance and registered participants who could not get in; the organisers stated that participants had been told in advance that seats were limited [17, 18].

Each repository was named `hack-<id>-<team>` and started with an organiser commit titled “Initial commit” that added a one-line README. The organisers archived the repositories after the deadline: GitHub’s `updated_at` timestamps place 3,201 archive operations between 18:01 and 18:51, and all 3,649 repositories were archived when we collected the data. The event offered 12 tracks, ten sector tracks and two special tracks, each with one industry or public-sector partner (Table 2).

4 Data and methods

Collection. On 24 September 2026 we listed all repositories of the organisation through the GitHub REST API, collected default-branch commit counts and commit times through GraphQL, and downloaded every root README. We then made mirror clones of all 3,649 repositories, which keep every branch, pull-request reference and the full history. All clones completed. The repositories were archived, so their content could no longer change.

Definitions. A *team commit* is any commit reachable from a branch, counted once per SHA, except the organisers’ template commit. We identify the template commit by a rule that does not depend on the author account: a root commit

Table 2: The 12 tracks with the active repositories matched to each, their median event-window commits, and the share with test files and with Python as primary language. Task descriptions are our summaries of the teams’ READMEs.

No.	Track	Partner	Task	Repos	Commits (median)	Tests (%)	Python (%)
01	Energy	Samruk-Kazyna	Agentic AI that forecasts wind-farm power output from weather data	74	25	90.5	83.8
02	Finance	Freedom	“Money Graph”: rebuild an organised group from a transaction network (AML)	115	21	87.0	74.8
03	Management	Halyk Bank	“Career Quest”: AI guide linking employee skills to growth steps	89	21	93.3	60.7
04	Telecom	Beeline	Agent deciding which subscribers get which tariff offer	46	18	76.1	95.7
05	Logistics	ekt.kz	Automatic supplier orders to replenish stock, approved by a manager	80	22	91.2	67.5
06	Creative ind.	Firebird	“#79-lite”: pick up to three event contractors and explain the choice	113	16	88.5	63.7
07	Education	AI Sana	Challenge hub: turn a business problem into a task for student teams	145	20	86.2	31.0
08	Innovation	Samruk-Kazyna	“Khattama”: minutes and action items from Kazakh, Russian or mixed meeting audio	65	17	89.2	87.7
09	Communications	Halyk Bank	“Voice Router”: a voice AI that routes calls in an insurance contact centre	29	31	79.3	72.4
10	Trade	ekt.kz	AI shopping assistant: products, analogues, prices, stock, cart	71	19	90.1	49.3
11	Special	Kazakhtelecom	Agent comparing org-structure documents before and after a reorganisation	42	20	92.9	61.9
12	Special	Astana Innovations	“Akim for five hours”: city simulator that splits a budget across areas and districts	115	23	94.8	30.4

titled “Initial commit” that changes one file by two added lines and was made within two minutes of the repository’s creation. An *empty repository* has no team commits. The *event window* is 13:00–18:00 local time on 23 September 2026, measured on commit timestamps; an *active repository* has at least one team commit in the window. A repository corresponds to one registration, which may be a team of up to three people or one person; we never infer head counts. RQ1 uses all 3,649 repositories, RQ2 and RQ7 the 1,057 active ones, and RQ3–RQ6 the 1,254 repositories with team commits.

Features. From the final state of each default branch we record languages by file size, dependencies declared in package manifests, and frameworks derived from them. We detect LLM SDKs from manifests and from import statements, excluding vendored folders. Agent traces are (i) context files: AGENTS.md (by that exact name), CLAUDE.md, and Cursor, Copilot and other agent rule files; (ii) commits whose co-author trailer, “generated with” line or author account names a coding agent; (iii) branches whose names start with an agent name; and (iv) pull-request references. For documentation we classify the root README by the letters of its prose after removing code, links and markup: Kazakh if at least three per cent of its Cyrillic letters are specific to Kazakh (nine Cyrillic letters that Russian does not use), Russian if Cyrillic letters are at least half of all letters, English if Latin letters are at least four fifths, and mixed otherwise. For credentials we ran Gitleaks [39] over every reference of every clone with redacted output, kept only the rule, file and time of each finding, and never tested any string against a service.

Track labels. Track labels come from an earlier per-repository table and were checked against README text with keyword rules; we read every disagreement and corrected 7 labels. 73 active repositories could not be assigned to a track.

Validation. Table 3 summarises the checks. Default-branch commit counts from the clones equal those from the GitHub API for every repository. The rebuilt table agrees with the earlier per-repository table for nearly all repositories; the few differences come from commits reachable only through pull-request references, which the earlier table counted and we report separately, and from repositories whose earlier values we could not reproduce. A live check of 25 sampled repositories against GitHub showed that some template commits had been made by a second organiser account and

Table 3: Validation checks and their results.

Check	Result
Default-branch commit counts, clone vs. GitHub API	3,649 of 3,649 identical
Template commit found by the account-independent rule	3,615 repositories; 34 rewrote history
Template commits by a second organiser account counted as team commits	16 repositories, corrected
Team commits per repository, rebuilt vs. earlier table	3,641 of 3,649 equal
Event-window commits per repository	3,644 of 3,649 equal
Hours with commits per repository	3,648 of 3,649 equal
Track labels read against README text	7 corrected; 73 unclear
OpenAI-pattern findings with the structural marker of OpenAI keys	77 of 77
README language rule vs. hand labels (90 READMEs)	[pending: labelling in progress]

were counted as team commits. We then audited all 402 repositories where this could affect the counts and found 16 such repositories, all created early and with test-like names; we corrected them.

Statistics. We wrote the analysis plan before running any test. Proportions carry Wilson 95% confidence intervals and medians bootstrap intervals from 10,000 resamples. We compare groups with Mann–Whitney tests (effect size Cliff’s δ) and χ^2 tests (Cramér’s V), with Holm correction within each question, and model activity by repository creation date with logistic regression. Work rhythms are clustered with k -means on each team’s hourly commit shares, with k from two to six chosen by silhouette score. All numbers in this paper are generated from the data by the released scripts.

Use of AI tools. The collection and analysis code, the figures and a first draft of this text were produced with an AI coding agent (Claude Code, Anthropic) under the author’s direction. The author set the research questions, definitions and analysis plan, and checked the outputs; the checks in Table 3 were designed to catch errors in AI-produced analysis, and they did. [pending: author to name any other AI tools used before this pipeline]

5 Results

5.1 RQ1: From registration to code

Of the 3,649 repositories, 1,254 (34.4%) received at least one team commit and 1,057 (29.0%) received team commits during the event; 2,395 (65.6%) received none (Figure 2a). Another 197 repositories have team commits only outside the window, almost all before the event. Of the active repositories, 984 (93.1%) could be assigned to a track.

The share of repositories that became active depended on when they were created (Figure 2b; $\chi^2(4) = 609.2$, $p < 0.001$, $V = 0.409$). It was 53.7% for repositories created between 1 September and 14 September and 34.3% for those created between 15 September and 21 September. On the evening of 22 September, 1,040 repositories were created within 18:00–20:00, up to 15 per minute, while on earlier days no minute saw more than 8. We read this as a batch created by the organisers, not as individual registrations; only 14 of these repositories (1.3%) became active. With the batch included, each week of earlier creation multiplied the odds of activity by 1.878 (95% CI 1.714–2.058); without it the effect is smaller, 1.170 (1.082–1.265, $n = 2,570$).

Some registrations appear to be repeated: 225 team names occur on more than one repository. Excluding the generic name “team”, 224 names cover 480 repositories, and in 146 of these groups exactly one repository became active.

5.2 RQ2: Work rhythm

Teams made 31,955 team commits, 30,418 of them in the event window. Activity rose in the first hour, stayed high through the afternoon, and peaked in the last hour, which holds 28.8% of event-time commits; the busiest ten minutes were 17:40–17:50 with 1,649 commits (Figure 3a). The median active team made its first commit 61 minutes after the start (95% CI 59–64) and its last commit 9 minutes before the deadline (8–10), with a median of 19 commits (interquartile range 10–35).

Most teams worked throughout: 79.3% committed in at least four of the five hours and 44.5% in all five, and 96.3% committed in the last hour. Clustering the hourly profiles gives two groups (silhouette 0.304; Figure 3b). A steady group of 734 teams (69.4%) spread its commits evenly after the first hour. A late-surge group of 323 teams (30.6%) made 58.3% of its commits in the last hour. After the deadline, 235 team commits from 74 repositories carry timestamps between 18:00 and 19:00.

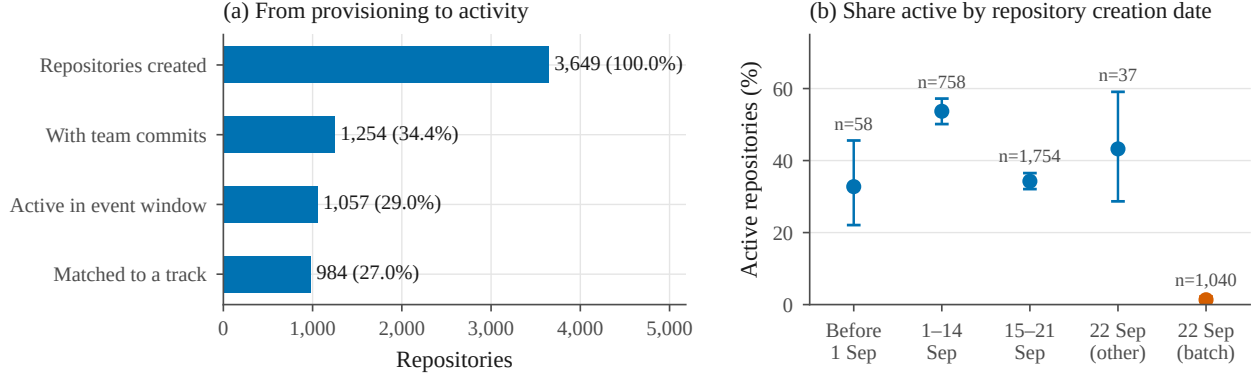


Figure 2: Participation. (a) Repositories at each stage. (b) Share of repositories that became active by creation date, with 95% Wilson intervals; the batch created on the evening before the event is shown separately.

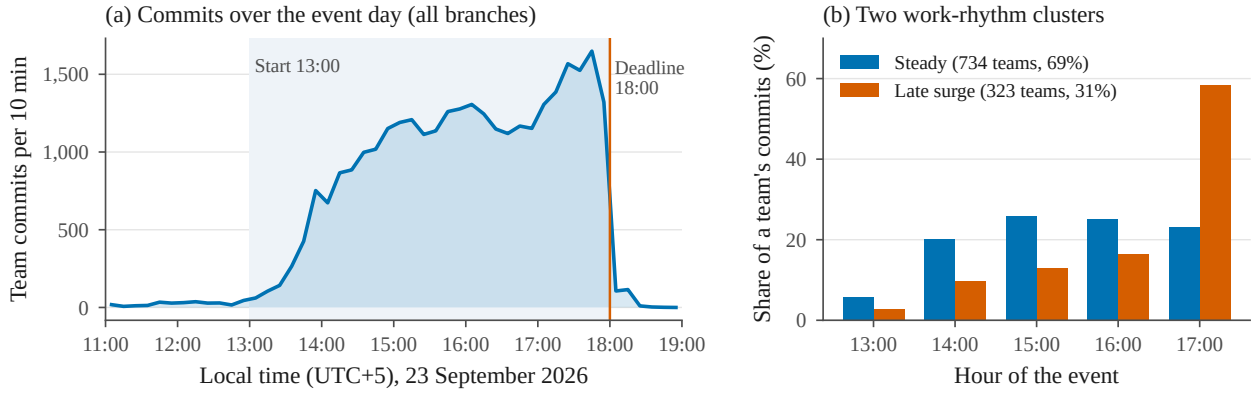


Figure 3: Work rhythm. (a) Team commits per ten minutes on the event day, all branches; the shaded band is the event window. (b) Mean hourly share of each team’s event-window commits in the two k -means clusters.

5.3 RQ3: Technology choices

Python was the primary language in 58.2% of the 1,254 repositories with team commits, followed by TypeScript (19.1%) and JavaScript (12.2%). The most common frameworks were React (40.2%), FastAPI (37.2%), Vite (28.2%) and Next.js (14.3%). An LLM SDK appeared in 63.7% of repositories, the OpenAI SDK in 60.0%, and other providers each in less than three per cent (Anthropic 2.6%, Ollama 2.3%, Google 2.2%). Engineering artefacts were common: test files in 76.9% of repositories (95% CI 74.5–79.1), a Dockerfile in 28.6%, a Compose file in 26.3%, a CI workflow in 12.4% and a deployment configuration in 5.3% (Figure 4). The primary language differed between tracks ($\chi^2(33) = 201.1$, $p < 0.001$, $V = 0.261$); for example, Python was the primary language in 95.7% of Telecom repositories and 30.4% of the city-simulator track (Table 2).

5.4 RQ4: Traces of coding agents

Agent context files were present in 31.0% of repositories: AGENTS.md in 28.7% (360) and CLAUDE.md in 12.2% (153); 36 repositories contained other agent rule files. 3,979 commits (12.5%) in 260 repositories carry an agent signature. Most signatures name Claude (3,502 commits) and few name Codex (333), although Codex was the required tool. Codex appears instead in branch names: 200 repositories have branches starting with codex, and 31.4% have pull-request references. We therefore treat commit signatures as evidence of each tool’s default disclosure behaviour, not as a measure of use.

Active repositories with an AGENTS.md file had more event-window commits (median 28 vs. 16; Cliff’s $\delta = 0.356$, Holm-adjusted $p < 0.001$), more hours with commits (median 5 vs. 4; $\delta = 0.260$, $p < 0.001$) and more often test files (95.7% vs. 82.7%; $V = 0.183$, $p < 0.001$). These are associations: teams that set up context files may differ from others in experience or preparation. The median non-merge commit changed 134 lines (interquartile range

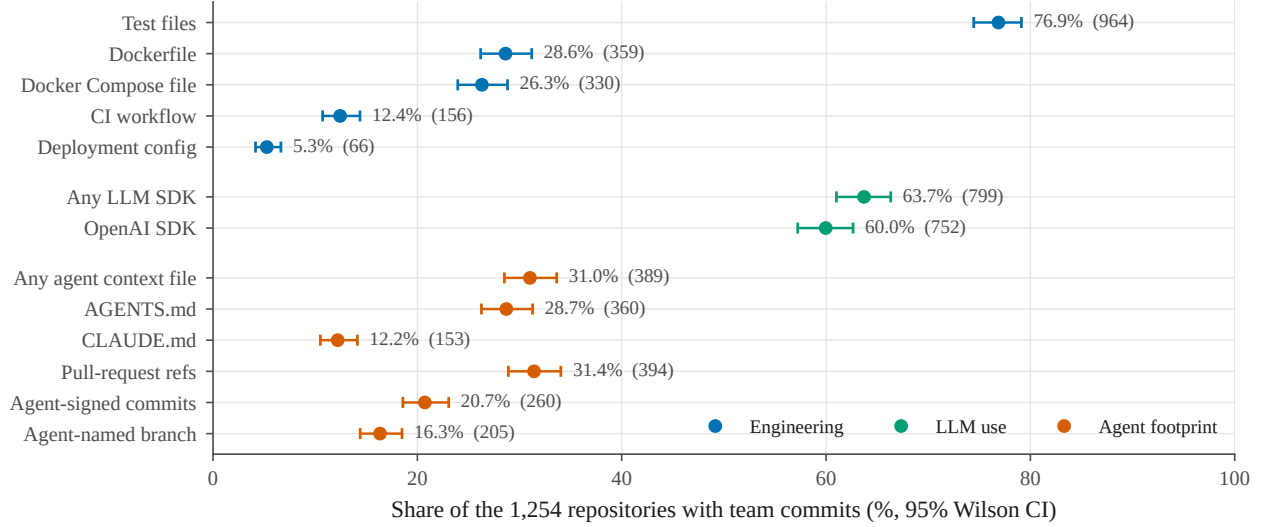


Figure 4: Engineering artefacts, LLM use and agent traces in the 1,254 repositories with team commits (share with 95% Wilson intervals; counts in brackets).

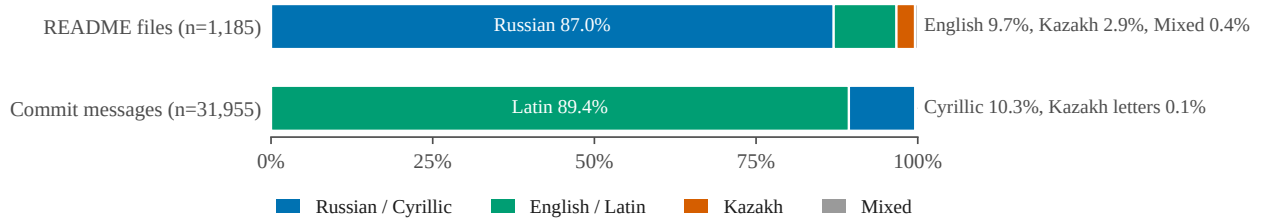


Figure 5: Language of README files and script of commit messages.

21–561), with similar medians for agent-signed and other commits (126 and 135); 36.6% of commit subjects follow the conventional-commit format.

5.5 RQ5: Documentation

Of the 1,185 READMEs with written content, 87.0% are in Russian, 9.7% in English, 2.9% in Kazakh and 0.4% mixed (Figure 5); another 63 repositories kept only the organiser’s one-line README. Kazakh-specific letters appear somewhere in 12.0% of READMEs. Commit messages are mostly in Latin script (89.4%), and Kazakh-specific letters occur in 23 of 31,955 commits. READMEs are long for a five-hour event (median 1,477 words, 95% CI 1,395–1,568; median 15 headings); 92.2% give run or installation instructions, 17.8% include images and 6.8% link to a deployed application.

5.6 RQ6: Credential hygiene

Gitleaks’ generic rule produced 4,597 findings in 131 repositories, 93.5% of them in data files such as the datasets provided with the tasks; we treat them as low-precision and do not report them further. Provider-specific rules produced 127 findings, of which 10 looked like placeholders. The remaining findings occur in 73 repositories (5.8% of those with team commits), and in 36 of them the string is still present in the final version (Figure 6a). Most are OpenAI API keys: 77 occurrences in 61 repositories (4.9%; 30 in the final version). All of them have the structural marker of OpenAI keys, and all were committed during the event window. We did not test whether any key worked.

The keys sat mainly in environment files (Figure 6b): 32 occurrences in `.env.example` and 27 in `.env`. The first is a template that projects commit on purpose, so a real key placed there becomes public even when `.gitignore` excludes `.env`. This matches the association we observe: keys were more frequent in repositories whose `.gitignore` covered `.env` (5.9% vs. 2.1%; $V = 0.079$, $p = 0.008$). Other hygiene signals were rare: a committed `.env` in 2.7%,

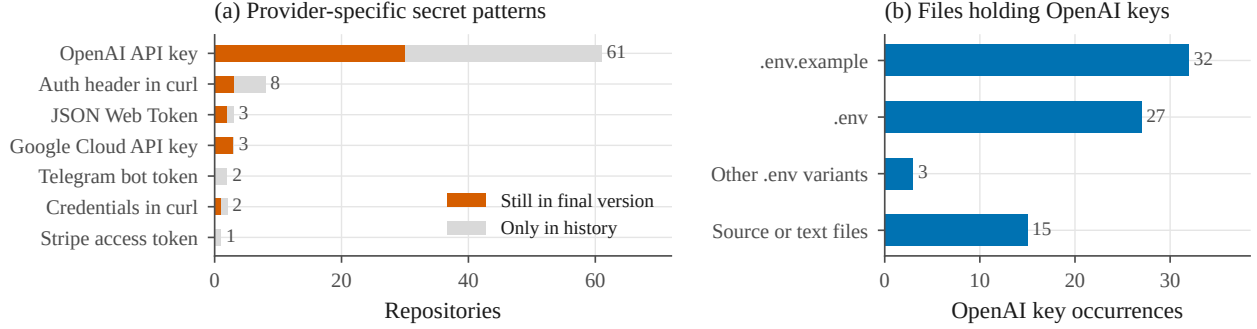


Figure 6: Credentials in public history. (a) Repositories with findings of provider-specific rules, split by whether the string remains in the final version. (b) Files that held OpenAI key strings.

committed `__pycache__` in 3.7%, `node_modules` in 0.6% and virtual environments in 0.4% of repositories; 73.0% had a `.gitignore` covering `.env`.

5.7 RQ7: Tracks

The education track drew the most active repositories (145), followed by finance (115) and the city simulator (115); the voice-routing track drew the fewest (29; Table 2). Event-window commits did not differ clearly between tracks (Kruskal–Wallis $H = 19.0$, $p = 0.062$), while the share with test files ranged from 76.1% (Telecom) to 94.8% (city simulator).

6 Discussion

Registrations are not participants. Two thirds of the repositories received no team commits. The data cannot tell whether a team did not come, could not enter, registered twice or worked elsewhere, and news reports describe people who registered but could not get in [17, 18]. The batch created on the evening before the event, of which almost none became active, suggests that repository creation and seat allocation were not coupled. For organisers, creating repositories only after attendance is confirmed would make the organisation a record of participants rather than of registrations; for researchers, organiser-created populations need this distinction before any rate is interpreted.

Working with agents under a deadline. The last-hour peak and the late-surge cluster resemble deadline behaviour described for developers in general [37], and they occur alongside steady work by most teams. Tests, Docker files and long READMEs were common in projects built in five hours. We have no baseline from comparable events without agents, so we cannot say how unusual this is; with agents writing much of the code in similar settings [22], such artefacts may be cheap to produce. We did not measure whether the tests run or pass, so their presence says nothing about quality, and quality risks of agent-written code are documented elsewhere [11, 32, 33].

Measuring agent use from repositories. Commit signatures named Claude far more often than Codex, although Codex was required. A plausible reason is that tools differ in whether they sign commits by default. Studies that estimate agent adoption from trailers or author accounts, as in datasets of agent pull requests [10], should expect such tool-specific bias; branch names, pull-request references and context files give complementary traces, none of them complete.

Language. Documentation was written mostly in Russian and English; Kazakh appeared in a small share of READMEs and almost never in commit messages. The data do not explain this. Possible factors include the audience teams wrote for, the language of the tasks, and the language in which teams worked with the agents.

Credentials. The keys we found were committed during the event, mostly to environment files, and some remain in archived repositories whose history cannot be changed by the teams. Simple measures would have prevented most of them: repository templates with a `.gitignore` that excludes environment files and an `.env.example` that holds only placeholders, organisation-level secret scanning with push protection, and short-lived keys issued for the event. These measures follow earlier recommendations [26, 29] and matter more when an agent writes configuration files quickly.

Lessons for mining organiser-created repositories. Three issues would have biased our results had we not checked them: template commits made by a second organiser account, commits reachable only through non-default branches or pull-request references, and one repository per registration rather than per person. As earlier work warns [34, 35], such details can change basic counts, and they are easy to miss when a first analysis is produced quickly.

7 Threats to validity

Construct validity. A repository is a registration, not a person or necessarily a team. Commit counts measure activity, not effort or quality. Agent traces are partial and tool-dependent. Secret findings are strings that match known key formats; we did not test them.

Internal validity. Track labels were checked by one person against README text. The README language rule is a heuristic based on letters [pending: accuracy against hand labels]. The event window is inferred from commit and archive times and press reports, not from an official schedule. Associations with AGENTS.md may reflect differences between teams rather than an effect of the file.

External validity. We study one event, in one country, with one required agent. Other events differ in duration, rules and participants.

Reliability. The pipeline, from collection to every number and figure, is released; the data it starts from are public repositories that are now archived.

8 Ethics, data availability and use of AI tools

All data come from public repositories and public web pages; we accessed no private system. We did not analyse names, email addresses or other personal data: author identities were only counted, never stored, and repository names are hashed in the released data. Secret values were redacted by the scanner, never stored or printed by our code, and never tested. We informed the organisers privately about the repositories containing credentials before publication [pending: confirm after contacting the organisers]. We are not affiliated with the organisers, and this paper does not report official results of the event.

The pipeline, the derived dataset with hashed identifiers, and the scripts that generate every number, table and figure are available at <https://github.com/tbayan/HackalemAi-Repo-Data-Mining> [pending: archived release with DOI]. AI tools were used as described in Section 4; the author takes responsibility for all content.

9 Conclusion

Because the organisers of HackAlem AI created a repository for every registration, the organisation records the full path from registration to code under a common deadline and a common agent requirement. Of 3,649 registrations, 1,057 produced code during the event. Most active teams worked through all five hours, a third of them concentrated their work at the end, and their repositories contain tests, Docker files, agent context files and long documentation, mostly in Russian. Strings with the structure of OpenAI keys reached the public history of 61 repositories. After the results are announced on 1 October 2026, we plan to relate these records to the jury’s decisions, and to compare rule-based and LLM-based extraction of repository features against human labels.

References

- [1] Alexander Nolte, Irene-Angelica Chounta, and James D. Herbsleb. What Happens to All These Hackathon Projects? Identifying Factors to Promote Hackathon Project Continuation. *Proceedings of the ACM on Human-Computer Interaction*, 4(CSCW2):1–26, October 2020. doi:10.1145/3415216.
- [2] Lukas McIntosh and Caroline D. Hardin. Do Hackathon Projects Change the World? An Empirical Analysis of GitHub Repositories. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education*, SIGCSE ’21, page 879–885. ACM, March 2021. doi:10.1145/3408877.3432435.
- [3] Sjoerd Halmans, Lavinia Paganini, Alexander Serebrenik, and Alexander Nolte. HackRep: A Large-Scale Dataset of GitHub Hackathon Projects. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR ’26, page 579–583. ACM, April 2026. doi:10.1145/3793302.3793305.

- [4] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590, 2023. URL <https://arxiv.org/abs/2302.06590>.
- [5] Albert Ziegler, Eirini Kalliamvakou, X. Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. Measuring GitHub Copilot’s Impact on Productivity. *Communications of the ACM*, 67(3):54–63, February 2024. doi:[10.1145/3633453](https://doi.org/10.1145/3633453).
- [6] Shraddha Barke, Michael B. James, and Nadia Polikarpova. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):85–111, April 2023. doi:[10.1145/3586030](https://doi.org/10.1145/3586030).
- [7] Jenny T. Liang, Chenyang Yang, and Brad A. Myers. A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE ’24, page 1–13. ACM, February 2024. doi:[10.1145/3597503.3608128](https://doi.org/10.1145/3597503.3608128).
- [8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-world GitHub Issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html. Preprint: doi:[10.48550/arXiv.2310.06770](https://doi.org/10.48550/arXiv.2310.06770).
- [9] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37*, NeurIPS 2024, page 50528–50652. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2024. doi:[10.52202/079017-1601](https://doi.org/10.52202/079017-1601).
- [10] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. arXiv preprint arXiv:2507.15003, 2025. URL <https://arxiv.org/abs/2507.15003>.
- [11] Shyam Agarwal, Hao He, and Bogdan Vasilescu. AI IDEs or Autonomous Agents? Measuring the Impact of Coding Agents on Software Development. In *Proceedings of the 23rd International Conference on Mining Software Repositories*, MSR ’26, page 857–862. ACM, April 2026. doi:[10.1145/3793302.3793589](https://doi.org/10.1145/3793302.3793589).
- [12] Wangyiyao Zhou, Alexander Serebrenik, and Alexander Nolte. Quick Build, Careful Check? Generative AI Use in Hackathons. In *Proceedings of the 2026 International Conference on Game Jams Hackathons and Game Creation Events*, ICGJ 2026, page 23–27. ACM, August 2026. doi:[10.1145/3833089.3833091](https://doi.org/10.1145/3833089.3833091).
- [13] Kiev Gama, Filipe Calegario, Victoria Jackson, Alexander Nolte, Luiz Morais, and Vinicius Cardoso Garcia. “Can you feel the vibes?”: An exploration of novice programmer engagement with vibe coding. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*, ICSE-SEET ’26, page 485–496. ACM, April 2026. doi:[10.1145/3786580.3786992](https://doi.org/10.1145/3786580.3786992).
- [14] Ashley J. Chen, Yijia Cao, Minghao Shao, Ramesh Karri, and Muhammad Shafique. Code for All: Educational Applications of the “Vibe Coding” Hackathon in Programming Education across All Skill Levels. arXiv preprint arXiv:2604.22747, 2026. URL <https://arxiv.org/abs/2604.22747>.
- [15] Ramteja Sajja, Carlos Erazo Ramirez, Zhouyayan Li, Bekir Z. Demiray, Yusuf Sermet, and Ibrahim Demir. Integrating Generative AI in Hackathons: Opportunities, Challenges, and Educational Implications. *Big Data and Cognitive Computing*, 8(12):188, December 2024. doi:[10.3390/bdcc8120188](https://doi.org/10.3390/bdcc8120188).
- [16] HackAlem AI. HackAlem AI — Astana, 23.09.2026, 2026. URL <https://hackalem.ai/>. Official event website. Accessed 24 September 2026.
- [17] Qumash.kz. Five hours in the queue: HackAlem AI participants in Astana complained about the organisation, September 2026. URL <https://qumash.kz/news/pyat-chasov-v-ocheredi-uchastniki-hackalem-a-i-v-astane-pozhalovalis-na-organizatsiyu/>. In Russian. Accessed 24 September 2026.
- [18] Ulys Media. “Survival simulator”: participants criticised HackAlem AI in Astana, September 2026. URL <https://ulysmedia.kz/news/81431-simuliator-vyzhivaniia-uchastniki-raskritikovali-hackalem-ai-v-astane/>. In Russian. Accessed 24 September 2026.
- [19] The Astana Times. Astana stages massive AI hackathon in Guinness World Records bid, September 2026. URL <https://astanatimes.com/2026/09/astana-stages-massive-ai-hackathon-in-guinness-world-records-bid/>. Accessed 24 September 2026.
- [20] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Lee-laprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent

- READMEs: An Empirical Study of Context Files for Agentic Coding. *ACM Transactions on Software Engineering and Methodology*, September 2026. doi:[10.1145/3840295](https://doi.org/10.1145/3840295).
- [21] Thibaud Gloaguen, Niels Mündler, Mark Müller, Veselin Raychev, and Martin Vechev. Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? arXiv preprint arXiv:2602.11988, 2026. URL <https://arxiv.org/abs/2602.11988>.
 - [22] Joachim Baumann, Vishakh Padmakumar, Xiang Li, John Yang, Diyi Yang, and Sanmi Koyejo. SWE-chat: Coding Agent Interactions From Real Users in the Wild. arXiv preprint arXiv:2604.20779, 2026. URL <https://arxiv.org/abs/2604.20779>.
 - [23] Muhammad Waseem, Aakash Ahmad, Kai-Kristian Kemell, Jussi Rasku, Sami Lahti, Kalle Mäkelä, and Pekka Abrahamsson. Vibe Coding in Practice: Flow, Technical Debt, and Guidelines for Sustainable Use. arXiv preprint arXiv:2512.11922, 2025. URL <https://arxiv.org/abs/2512.11922>.
 - [24] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*, ITiCSE 2023, page 108–159. ACM, December 2023. doi:[10.1145/3623762.3633499](https://doi.org/10.1145/3623762.3633499).
 - [25] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. Computing Education in the Era of Generative AI. *Communications of the ACM*, 67(2):56–67, January 2024. doi:[10.1145/3624720](https://doi.org/10.1145/3624720).
 - [26] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories. In *Proceedings 2019 Network and Distributed System Security Symposium*, NDSS 2019. Internet Society, 2019. doi:[10.14722/ndss.2019.23418](https://doi.org/10.14722/ndss.2019.23418).
 - [27] Vibha Singhal Sinha, Diptikalyan Saha, Pankaj Dhoolia, Rohan Padhye, and Senthil Mani. Detecting and Mitigating Secret-Key Leaks in Source Code Repositories. In *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, page 396–400. IEEE, May 2015. doi:[10.1109/msr.2015.48](https://doi.org/10.1109/msr.2015.48).
 - [28] Setu Kumar Basak, Lorenzo Neil, Bradley Reaves, and Laurie Williams. SecretBench: A Dataset of Software Secrets. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, page 347–351. IEEE, May 2023. doi:[10.1109/msr59073.2023.00053](https://doi.org/10.1109/msr59073.2023.00053).
 - [29] Alexander Krause, Jan H. Klemmer, Nicolas Huaman, Dominik Wermke, Yasemin Acar, and Sascha Fahl. Pushed by Accident: A Mixed-Methods Study on Strategies of Handling Secret Information in Source Code Repositories. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2527–2544, Anaheim, CA, August 2023. USENIX Association. ISBN 978-1-939133-37-3. URL <https://www.usenix.org/conference/usenixsecurity23/presentation/krause>. Preprint: doi:[10.48550/arXiv.2211.06213](https://doi.org/10.48550/arXiv.2211.06213).
 - [30] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. Your Code Secret Belongs to Me: Neural Code Completion Tools Can Memorize Hard-Coded Credentials. *Proceedings of the ACM on Software Engineering*, 1(FSE):2515–2537, July 2024. doi:[10.1145/3660818](https://doi.org/10.1145/3660818).
 - [31] Pinran Gao, Lingxiang Wang, Yi Liu, Kunpeng Liu, Fan Yang, and Ying Zhang. Mind your key: An Empirical Study of LLM API Credential Leakage in iOS Apps. arXiv preprint arXiv:2606.12212, 2026. URL <https://arxiv.org/abs/2606.12212>.
 - [32] Songwen Zhao, Danqing Wang, Kexun Zhang, Jiaxuan Luo, Zhuo Li, and Lei Li. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks. arXiv preprint arXiv:2512.03262, 2025. URL <https://arxiv.org/abs/2512.03262>.
 - [33] Junquan Deng, Zhiyu Fan, and Ruijie Meng. Understanding the (In)Security of Vibe-Coded Applications. arXiv preprint arXiv:2606.23130, 2026. URL <https://arxiv.org/abs/2606.23130>.
 - [34] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. The promises and perils of mining GitHub. In *Proceedings of the 11th Working Conference on Mining Software Repositories*, ICSE ’14, page 92–101. ACM, May 2014. doi:[10.1145/2597073.2597074](https://doi.org/10.1145/2597073.2597074).
 - [35] Nicole Hoess, Carlos Paradis, Rick Kazman, and Wolfgang Mauerer. Does the Tool Matter? Exploring Some Causes of Threats to Validity in Mining Software Repositories. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, page 645–656. IEEE, March 2025. doi:[10.1109/saner64311.2025.00067](https://doi.org/10.1109/saner64311.2025.00067).
 - [36] Gede Artha Azriadi Prana, Christoph Treude, Ferdian Thung, Thushari Atapattu, and David Lo. Categorizing the Content of GitHub README Files. *Empirical Software Engineering*, 24(3):1296–1327, October 2018. doi:[10.1007/s10664-018-9660-3](https://doi.org/10.1007/s10664-018-9660-3).

-
- [37] Zeinabsadat Saghi, Thomas Zimmermann, and Souti Chattopadhyay. Code Today, Deadline Tomorrow: Procrastination Among Software Developers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, page 1204–1216. IEEE, April 2025. doi:[10.1109/icse55347.2025.00198](https://doi.org/10.1109/icse55347.2025.00198).
 - [38] Kazinform. What is built in five hours: how the largest AI hackathon in Astana is run, September 2026. URL <https://www.inform.kz/ru/chto-sozdayut-za-pyat-chasov-kak-prohodit-krupneyshiy-ai-hakaton-v-astane-72e380f9>. In Russian. Accessed 24 September 2026.
 - [39] Gitleaks contributors. Gitleaks: a tool for detecting secrets in git repositories, version 8.30.1, 2026. URL <https://github.com/gitleaks/gitleaks>. Release v8.30.1, 21 March 2026. Accessed 24 September 2026.